MPTA-Repair: Clock-Bound Repair for Multi-Property Timed Automata in Industrial Practice

***Abstract*—Timed automata (TAs) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple predefined properties simultaneously. When verification fails, repairing the model is challenging because it requires modifying erroneous clock constraints to satisfy all properties while preserving functional equivalence. To address this, we present MPTA-Repair, an automated repair framework for multi-property TAs. The framework employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. To optimize search efficiency, it systematically exploits relationships among properties, counterexamples, and repair variables. Each candidate model is then validated through full-property regression verification and acceptability checks. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks derived from UPPAAL and the XtaBenchmarkSuite, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—Timed automata, automated model repair, MaxSMT, multi-property verification**

I. Introduction

Timed automata (TAs) are widely used to model and verify real-time systems whose correctness depends on both discrete behavior and quantitative timing constraints [1], [2]. They appear in railway control, communication protocols, medical cyber-physical systems, and aerospace scheduling [3]–[5]. In these systems, an alarm, message, pulse, actuation, or resource release may be functionally present but still incorrect if it occurs too early, too late, or for an invalid duration.

Practical timed-automata models are validated against sets of properties rather than isolated assertions. These properties jointly capture safety, bounded response, timeout handling, deadline satisfaction, alarm duration, and deadlock freedom [6]. When a property is violated, model checkers such as UPPAAL and Kronos can produce a timed diagnostic trace (TDT), i.e., a timed counterexample that reaches a violating state [7], [8]. A TDT is useful failure evidence, but it does not identify which numerical bound is wrong, whether the fault lies in a guard or an invariant, or how the bound should be changed.

The repair problem becomes harder in the multi-property setting. A bound change that blocks one diagnostic trace may leave another violation feasible, break a property that was previously satisfied, or disable intended untimed behavior. The running example in Section II illustrates this effect: repairing an early gear-engagement violation can make an acknowledgement deadline fail. A practical repair method must therefore generate candidate edits from local counterexamples, but accept them only after complete property regression and behavior-preservation checks.

Existing repair techniques provide an important foundation. Clock-bound repair encodes a TDT as linear real arithmetic, introduces variation variables for numerical clock bounds, and uses MaxSMT solving to compute small changes to guards and invariants [9]. TARTAR extends this idea with syntactic repair generation and an admissibility check based on untimed functional equivalence [10]. Related work also studies parametric timed automata, clock-guard repair, robustness analysis, and SMT-based repair [11]–[14]. However, TDT-based workflows are still commonly driven by one violated property or one diagnostic trace at a time.

This paper presents **MPTA-Repair**, an automated framework for multi-property clock-bound repair of timed automata. Given a faulty model, a property set, and diagnostic traces, MPTA-Repair searches for small changes to numerical bounds in transition guards and location invariants. It does not modify property formulas, locations, transitions, synchronization labels, clock references, reset sets, or data updates. The framework maintains a pool of property-trace obligations, uses relations among properties, counterexamples, and repairable bounds to guide MaxSMT-based candidate generation, and validates each candidate by full-property regression and a TARTAR-style admissibility check.

We evaluate MPTA-Repair on faulty models generated by clock-bound mutation. The evaluation includes benchmark models derived from UPPAAL and the XtaBenchmarkSuite, together with control-domain case-study models reconstructed from practical timed-automata examples. Compared with an iterative single-property baseline under the same editable clock-bound sites and global acceptance gates, MPTA-Repair improves repair success from 72% to 93% on the benchmark dataset and from 20% to 76% on the control-domain dataset.

Contributions. This paper makes four contributions: it formulates clock-bound repair as a multi-property and multi-counterexample problem; presents an iterative MaxSMT-based repair workflow guided by property, trace, and repair-variable relations; implements a prototype combining trace analysis, candidate generation, full-property regression, and admissibility checking; and evaluates the approach against an iterative single-property baseline while reporting practical lessons on

regression validation, admissibility, and repair-space limitations.

The remainder of the paper is organized as follows. Section II defines the problem setting. Section III presents MPTA-Repair. Section IV reports the implementation and evaluation. Section V concludes the paper.

II. Background and Motivation

A. Timed Automata and Repair Scope

Definition 1. A timed automaton is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I),$$

where L is a finite set of locations, ℓ_0 is the initial location, C is a finite set of clocks, Σ is a set of action labels, Θ is a finite set of transitions, and I maps each location to a clock invariant. A clock constraint is a conjunction of atoms $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and $b \in \mathbb{R}_{\geq 0}$. A transition $\theta = (\ell, g, a, R, \ell')$ moves from ℓ to ℓ' when guard g is satisfied, emits action a, and resets clocks in $R \subseteq C$. A network timed automaton is a parallel composition $\mathcal{N} = T_1 \parallel \cdots \parallel T_k$ with the synchronization semantics of the target modeling language, such as UPPAAL [7]. Delay steps must respect location invariants, whereas discrete steps are enabled by guards.

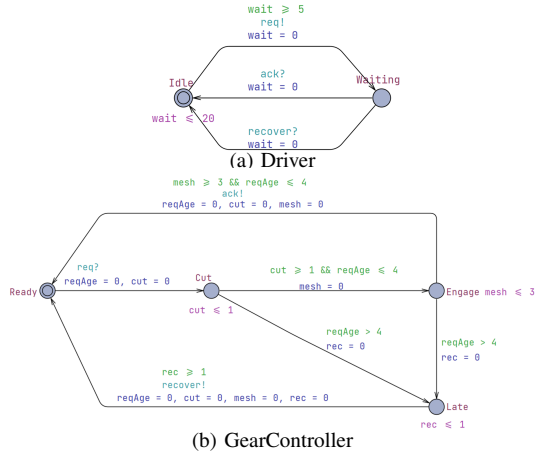
MPTA-Repair treats the network structure as fixed. The only repairable sites are numerical bound occurrences in transition guards and location invariants. A site s containing $x \sim b_s$ may be changed to $x \sim (b_s + \delta_s)$, where δ_s is a variation variable. The operator, clock reference, transition, location, synchronization label, reset set, data update, and property formula are not changed. This scope targets timing-parameter faults such as incorrect deadlines, timeouts, sampling periods, alarm durations, or minimum residence times; faults requiring reset, synchronization, data, or structural edits are outside the current repair space.

B. Diagnostic Traces and Multi-Property Motivation

Timed-automata verification in this paper is reduced to safety-style obligations. Direct examples include queries of the form $\mathrm{A}[]\,\psi$ and $\mathrm{A}[]\,not\ deadlock$. Bounded-response or deadline requirements are handled when monitor or observer templates encode them as reachable error locations [6].

Definition 2. A symbolic timed trace (STT) of a model is a finite transition sequence $S = \theta_0, \ldots, \theta_{n-1}$. A realization of S is a sequence of non-negative delays $d_0, \ldots, d_n$ and states $s_0, \ldots, s_{n+1}$ such that, for every $i < n$, the model can delay from s_i by d_i and then take θ_i to s_{i+1} while respecting guards, resets, invariants, and synchronizations; the final delay d_n leads from s_n to s_{n+1}. The STT is feasible if it has at least one realization. Given a safety property φ, a feasible STT is a timed diagnostic trace (TDT) for φ if at least one realization reaches a state that violates φ.

Example 1. Figure 1 shows the running gear-shift example. The Driver issues a request through $req!$, and the controller responds through $ack!$ or $recover!$. In the faulty controller, $cut \leq 1$ and $cut \geq 1$ allow engagement after one time unit,



(a) Driver

(b) GearController

Fig. 1. Simplified gear-shift example.

violating property φ_1, which requires at least two time units of torque cut. A single-property repair can change these bounds to 2. However, if the $mesh \leq 3$ and $mesh \geq 3$ bounds remain unchanged, the earliest acknowledgement occurs after $2 + 3 = 5$ time units, violating property φ_2, which requires acknowledgement within four time units. A multi-property repair must therefore adjust both cut and $mesh$ while preserving the same untimed request–engage–acknowledge behavior.

A TDT is failure evidence, not a root-cause explanation. It does not determine which numerical bound is faulty or whether the responsible occurrence is a guard or an invariant. TDT-based clock-bound repair encodes trace feasibility and property violation as linear arithmetic constraints over delays, valuations, and bound variations [9]. MaxSMT solving then prefers small repairs, typically by minimizing the number and magnitude of nonzero variations [15]. In the single-trace setting, the resulting candidate blocks the considered violating realization, but it does not prove that the whole model is repaired.

Definition 3. Let M_{bad} be a faulty timed-automata network and let $\Phi = \{\varphi_1, \ldots, \varphi_m\}$ be the complete property set used for validation. A clock-bound repair constructs M_{rep} by changing a small set of repairable bound occurrences. The repair is accepted only if the repaired bounds are well formed,

$$M_{rep} \models \Phi \quad \text{and} \quad \mathrm{Untimed}_{\Sigma_f}(M_{bad}) = \mathrm{Untimed}_{\Sigma_f}(M_{rep}).$$

Here Σ_f is the functional alphabet used for admissibility after excluding monitor-only, observer-only, and internal labels when appropriate. This TARTAR-style criterion prevents repairs that satisfy properties merely by disabling required actions [10].

The objective in Definition 3 explains why MPTA-Repair accumulates property-trace obligations across refinement rounds. A bound edit that fixes one TDT may leave another violation feasible, break a previously satisfied property, or remove intended functionality. MPTA-Repair therefore uses TDT encodings for candidate generation only, and accepts a candidate only after full-property regression and functional-admissibility checking.

III. Approach

A. Overview

MPTA-Repair is a counterexample-guided framework designed for the multi-property clock-bound repair of timed automata models. The framework takes as input a faulty model M, a property set $\Phi = \{\varphi_1, \ldots, \varphi_m\}$, and configuration parameters such as a global timeout. Instead of requiring users to manually specify repair variables, MPTA-Repair extracts repairable bounds from numerical clock constraints in transition guards and location invariants. The output is either a repaired model M' along with a set of clock-bound edits, or a failure report indicating that no admissible full-property repair was identified within the given time budget.

MPTA-Repair is based on two observations. First, while candidate repair generation can be driven by local diagnostic traces, candidate validation must remain global. A candidate that repairs one violated property might induce failures in others, because shared clocks, locations, transitions, or synchronizations often participate in multiple timing requirements. Consequently, MPTA-Repair iteratively collects counterexamples from subsequent violations to progressively constrain the space of admissible repair candidates. This refinement loop continues until a candidate satisfies the entire property set and passes admissibility verification, or the given time budget is exhausted.

Second, different properties and diagnostic traces are often related through shared repairable bounds. A single erroneous timing bound can contribute to multiple violations, and distinct traces may correspond to different executions of the same underlying fault. To leverage this interdependence, MPTA-Repair maintains a joint pool of property-trace obligations and searches for a clock-bound assignment that simultaneously satisfies all encoded constraints while minimizing the total modification cost. Relational analysis across properties, traces, and repairable bounds serves to minimize redundant computations and guide the search space exploration, though the generated candidates must ultimately pass verification and admissibility checks.

In operation, MPTA-Repair initially verifies the model against the property set and collects timed diagnostic traces (TDTs) for any violations. It then extracts repairable clock bounds and constructs a joint MaxSMT repair problem from the active property-trace pool. The computed assignment is instantiated into a candidate model, which undergoes validation against the complete property set and a functional-equivalence admissibility criterion. If validation fails, newly identified diagnostic traces are integrated into the pool, and the refinement loop repeats.

Figure 2 illustrates the overall workflow. The initial verification stage populates the property-trace pool. Subsequently, the relation-guided optimization phase structures the dependencies among properties, counterexamples, and repairable bounds. Finally, the MaxSMT solving stage generates candidate edits. The validation component either accepts an admissible repair or feeds new diagnostic evidence back into the subsequent refinement round.

B. Joint Trace Encoding and Optimization Objective

MPTA-Repair adopts the bound-variation paradigm for clock-bound repair [9]. Each modifiable numerical bound b_s in a transition guard or location invariant is associated with a variation variable δ_s. This ensures that the repaired constraint retains the original clock reference and comparison operator while incorporating a shifted value:

$$b'_s = b_s + \delta_s.$$

The repair space is therefore strictly restricted to clock-bound constants, keeping transition structures and discrete updates unaltered.

To formalize path constraints, MPTA-Repair adapts the trace-encoding formulation of TARTAR [9]. Given a timed diagnostic trace

$$\tau = \ell_0 \xrightarrow{d_0, e_0} \ell_1 \cdots \xrightarrow{d_{n-1}, e_{n-1}} \ell_n,$$

a violation trace for property φ is encoded as a conjunction of linear arithmetic constraints:

$$\begin{aligned}\mathrm{Enc}(\tau, \varphi, \theta) = {} & \mathit{Init}(\nu_0) \\ & \wedge \bigwedge_{i=0}^{n-1} \Big(d_i \geq 0 \wedge \mathit{Inv}(\ell_i, \nu_i + d_i, \theta) \\ & \qquad \wedge \mathit{Guard}(e_i, \nu_i + d_i, \theta) \wedge \mathit{Reset}_i \Big) \\ & \wedge \mathit{Bad}_\varphi(\ell_n, \nu_n).\end{aligned}$$

Here, $\mathit{Init}(\nu_0)$ initializes the clock valuations, while d_i, Inv, Guard, and Reset_i define the non-negative delays, location invariants, transition guards, and clock resets at step i, respectively, under the modified bounds θ. The predicate $\mathit{Bad}_\varphi(\ell_n, \nu_n)$ characterizes the violation state of φ at the final location. To eliminate this violating behavior while preserving executability of the discrete path skeleton, the single-trace repair obligation eliminates the vector of trace-specific delay and valuation variables $\mathbf{z}_\tau$ via a quantifier elimination operator QE:

$$\Psi_{\tau,\varphi}(\theta) = \mathrm{QE}\left(\neg \exists \mathbf{z}_\tau .\ \mathrm{Enc}(\tau, \varphi, \theta)\right).$$

We extend this single-trace formulation to a joint, multi-property incremental setting. At refinement round k, counterexamples from all currently violated properties are accumulated into a trace pool

$$\mathcal{T}_k = \{(\tau_j, \varphi_j)\}_{j=1}^{m_k}.$$

MPTA-Repair assembles a joint MaxSMT problem with the constraint system defined as

$$\begin{aligned}\Phi_k(\theta) = {} & \mathit{Dom}(\theta) \wedge \mathit{Block}_k(\theta) \\ & \wedge \bigwedge_{(\tau,\varphi) \in \mathit{Select}(\mathcal{T}_k)} \Psi_{\tau,\varphi}(\theta).\end{aligned}$$

Here, $\mathit{Dom}(\theta)$ enforces static interval consistency and domain limits, such as $b'_s \geq 0$, and $\mathit{Select}(\mathcal{T}_k)$ represents a reduced

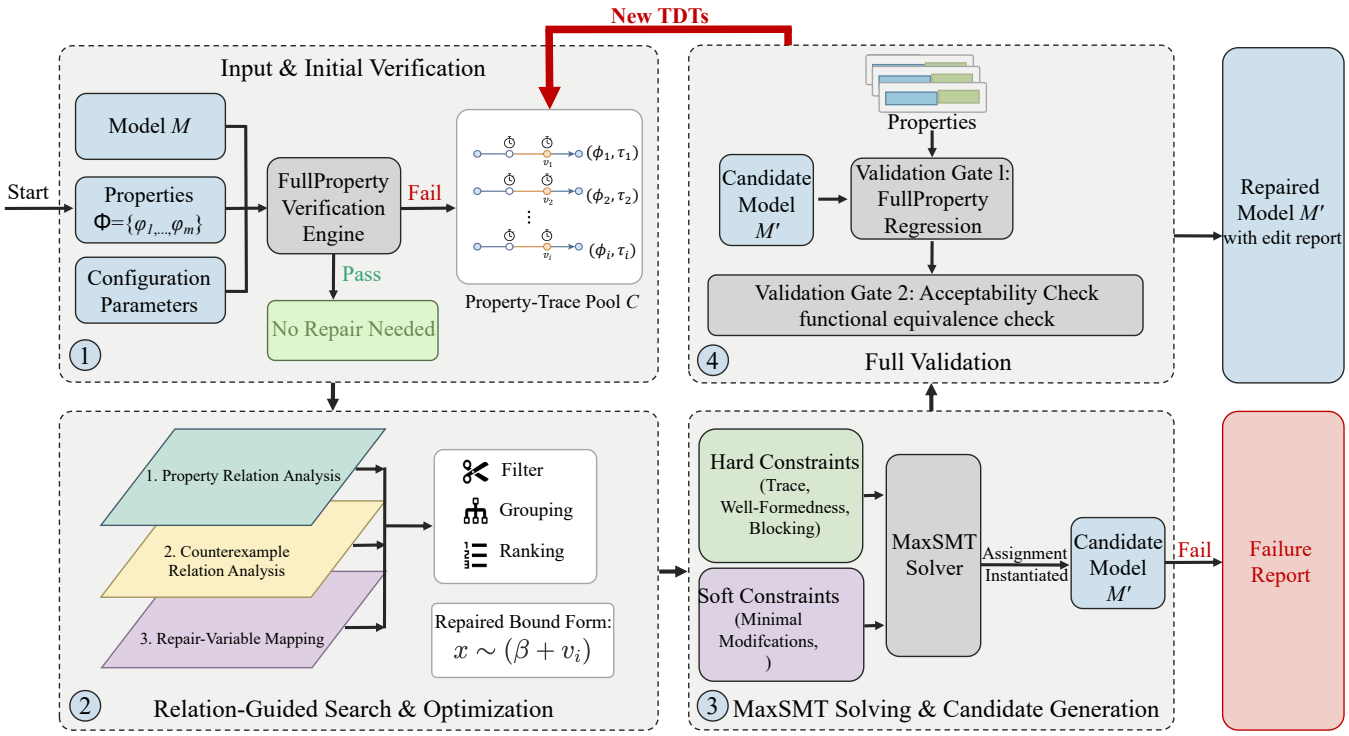


Fig. 2. Overview of the MPTA-Repair workflow.

subset of traces obtained after relation-aware pruning. When an instantiated candidate model fails verification, the constraint system is updated incrementally. The framework expands the pool via

$$\mathcal{T}_{k+1} = \mathcal{T}_k \cup NewTDT(M(\theta_k), \Phi)$$

and updates the search-space block to exclude the failed assignment θ_k over the set of repairable bounds S:

$$Block_{k+1}(\theta) = Block_k(\theta) \wedge \left(\bigvee_{s \in S} \delta_s \neq \delta_s^k \right),$$

where δ_s^k is the valuation assigned to variation variable δ_s in round k.

The optimization objective is governed by a hierarchical set of soft constraints. MPTA-Repair implements a lexicographic objective to prioritize engineering preferences and domain heuristics:

$$\operatorname*{lex\,min}_{\theta} \begin{pmatrix} \sum_{s \in S} \mathbf{1}\{\delta_s \neq 0\}, \\ \sum_{s \in S} |\delta_s|, \\ \sum_{s \in S} risk_s \cdot \mathbf{1}\{\delta_s \neq 0\}, \\ -\sum_{s \in S} benefit_s \cdot \mathbf{1}\{\delta_s \neq 0\} \end{pmatrix}.$$

Here, $\mathbf{1}\{\cdot\}$ denotes the indicator function, yielding 1 when the inner condition holds and 0 otherwise. The primary and secondary objectives minimize the number and total magnitude of altered clock bounds to produce concise repairs. The third term incorporates a penalty coefficient $risk_s$ to discourage modifications prone to regression, while the final term uses a reward coefficient $benefit_s$ to prioritize repair sites that cover a broader set of violated traces.

C. Relation-Guided Search Optimization

During successive refinement rounds, the accumulation of violated properties, diagnostic traces, and editable clock bounds significantly expands the search space [16]. Unstructured encoding of all property-trace constraints and repairable bounds increases the complexity of the MaxSMT problem and can lead to repeatedly solving similar constraints. To mitigate this overhead, MPTA-Repair introduces relation-guided optimization to filter, group, and rank repairable variables and constraints. Crucially, these relation-based heuristics serve strictly as search optimizations; validity and admissibility of generated candidates are still guaranteed through full-property regression verification and admissibility checking.

At the property layer, MPTA-Repair operates on timed safety properties normalized into an $\mathtt{A}[]$ fragment over location predicates and clock or integer constraints [6]. Relational analysis employs conservative syntactic and structural rules to evaluate equivalence and dominance. Two properties are defined as equivalent if and only if their normalized representations are syntactically identical. Dominance is established only when clear logical subsumption holds, for example when two upper-bound properties share identical structures but differ in their numerical thresholds and the tighter bound implies the weaker one. For instance, $\mathtt{A}[]\, x \leq 5$ subsumes $\mathtt{A}[]\, x \leq 10$ under

the same structural context. Consequently, MPTA-Repair can retain only the stronger properties for the subsequent solving phase to optimize efficiency.

At the counterexample layer, the framework maintains a managed pool of timed diagnostic traces (TDTs) indexed by their corresponding violated properties. Redundant traces are eliminated via exact-duplicate removal. The remaining TDTs are annotated with structural features, including traversed transitions, visited locations, active clocks, guard or invariant bounds, and violated properties from which they are generated [9]. Traces sharing identical discrete transition sequences or overlapping clock zones are grouped together to expose the common structural bottlenecks of the model. A trace constraint is omitted from the active encoding only when it is an exact duplicate or when a sound subsumption check shows that its local constraint pattern is already covered by an encoded trace constraint.

At the repair-bound layer, MPTA-Repair establishes a mapping between each active property-trace constraint and the clock bounds that occur in or affect the encoded trace constraints. This mapping restricts the set of active decision variables in the MaxSMT problem to those directly relevant to the current counterexample pool, while providing a heuristic ordering for candidate generation. Rather than imposing rigid structural constraints or hard-coding specific modifications, these relational scores serve as an adaptive guidance mechanism. They improve repair efficiency by ranking the repair variables, prioritizing those whose modifications are more likely to satisfy the joint constraint system.

D. Validation, Admissibility, and Refinement

In MPTA-Repair, candidate validation determines whether a generated edit set can be accepted as a repair [17]. After a candidate edit set is generated, the repaired model is verified against the entire property set Φ. If a candidate resolves the observed violation but induces new violations against the specification, it is rejected, indicating that the current property-trace pool is insufficient. MPTA-Repair then extracts the newly observed diagnostic trace and appends it to the pool, guiding the subsequent refinement round.

Candidates that pass full-property regression verification are further evaluated against an admissibility check to maintain functional equivalence [9]. This step ensures that clock-bound modifications do not satisfy properties by disabling original functional behaviors. Rather than enforcing timed-language equivalence, which would conflict with the goal of eliminating violating timed paths, admissibility is verified by constructing untimed automata from both the original and repaired models and comparing their accepted untimed languages [18]. A candidate is rejected if it adds or removes discrete functional paths, ensuring that only timing characteristics are modified.

The refinement loop therefore combines local candidate generation with global validation. Local MaxSMT formulations propose candidate assignments, full-property regression ensures complete specification compliance, and functional-equivalence checking maintains the original functional equivalence.

IV. Implementation and Experimental Evaluation

This section presents the implementation of MPTA-Repair and evaluates its performance on both benchmark and industrial case studies. To ensure practical utility, our approach focuses exclusively on repairing numerical clock bounds within transition guards and location invariants, rather than altering the discrete transition logic. This parametric repair space is suitable for industrial systems, as it corrects clerical errors and optimizes timing resources without disrupting the underlying functional design. In addition, comparing MPTA-Repair with an iterative baseline extended from the TARTAR paradigm [10] demonstrates that multi-property regression and multi-counterexample analysis are essential to prevent secondary errors.

A. Prototype Implementation

We implemented MPTA-Repair as an incremental pipeline that takes a faulty model and produces a validated repaired model or a failure report. The tool integrates a timed-automata parser, a verification interface using UPPAAL [7], a Z3-powered MaxSMT backend [19], and an admissibility checker. Modeled after a control-loop architecture, the pipeline executes the following steps.

Input and initial verification. The pipeline reads the faulty model, properties, and configurations. The verification engine extracts editable clock-bound variables from transition guards and location invariants, generating an initial trace pool of timed diagnostic traces for all violated properties.

Relation-guided search and optimization. This module maps the diagnostic traces to the extracted variables. A relation-analysis mechanism prunes, groups, and ranks these variables by identifying logical dependencies among properties, counterexamples, and repair locations. This mapping structures the repair constraints and reduces the search space.

MaxSMT solving and candidate generation. The optimized data is passed to the MaxSMT solver, which constructs a constraint system. This system includes hard constraints for trace feasibility, well-formedness, and blocking criteria, along with soft constraints designed to minimize modifications. Once a solution is found, a model writer generates the candidate model.

Full validation and refinement. The candidate model must pass two validation steps to ensure timing correctness and behavioral equivalence. First, full-property regression verification through UPPAAL ensures that no new violations are introduced. Second, an acceptability check evaluates untimed functional equivalence using a TARTAR-style criterion [9], rejecting candidates that alter the functional, untimed behavior.

Iteration. If a candidate fails either validation step, the control loop feeds the newly discovered diagnostic traces back into the trace pool. This initiates a counterexample-guided refinement loop, updating the solver constraints until a globally valid repair is found or the search budget is exhausted.

TABLE I
STATISTICS OF THE BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Original models	Mutated models	Avg. properties/model
Benchmark	9	54	4.1
Industrial	4	105	7.3

TABLE II
REPAIR SUCCESS RATES ACROSS BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Method	Success rate
Benchmark	Iterative baseline	72%
Benchmark	MPTA-Repair	93%
Industrial	Iterative baseline	20%
Industrial	MPTA-Repair	76%

B. Evaluation Strategy and Datasets

To evaluate MPTA-Repair, we use two datasets with different characteristics, summarized in Table I. The benchmark dataset includes nine standard models from UPPAAL and the XtaBenchmarkSuite, such as Elevator and FDDI, providing structural diversity for baseline comparison. The industrial dataset contains four closely coupled industrial case studies: Gear Control System for automotive transmission, SAE RoR for autonomous-driving decisions, Train Controller Alarm for railway emergency timing, and Train Gate Controller for shared-resource concurrency. Unlike the benchmarks, these industrial models contain dense timing constraints across multiple properties.

Since real-world faulty timed automata are rare, we synthesize faulty instances using mutation [20]. Fault injection modifies only numerical clock-bound constants in guards and invariants, excluding structural changes. Simple mutants contain a single localized modification, while complex mutants combine two non-redundant single-point faults. We filter these combinations to ensure that both faults contribute to the property violation. All retained mutants are syntactically valid, violate at least one timing safety specification, and pass an initial admissibility check to ensure their original untimed functional behavior is preserved before repair.

We compare MPTA-Repair against an adapted iterative TARTAR-style baseline [10]. For fairness, this baseline uses the same regression and admissibility validation steps as our method. The primary difference is that the baseline derives repairs from only one property or diagnostic trace at a time. Because it lacks the joint multi-counterexample relation analysis used in our framework, the baseline often gets stuck in local optima and overfits to isolated fault paths. This comparison highlights the importance of analyzing properties, counterexamples, and repair variables simultaneously.

C. Experimental Results

The experimental evaluation demonstrates that MPTA-Repair outperforms the iterative baseline, particularly on complex industrial models. As shown in Table II, the baseline achieves a 72% success rate on the benchmark dataset, whereas MPTA-Repair reaches 93%. This performance gap widens on the industrial dataset, where the baseline repairs only 20% of the cases compared to 76% for MPTA-Repair. These results indicate that our approach remains effective on standard timed-automata benchmarks while demonstrating robustness in complex industrial systems where properties and traces interact closely.

The baseline is competitive on simpler benchmark models because timing errors are frequently concentrated on isolated guards or invariants. In these localized scenarios, a single diagnostic trace often provides sufficient information to infer a correct repair. However, even within standard benchmarks featuring shared multi-channel structures, such as the FDDI protocol, MPTA-Repair is more robust because it prevents single-trace overfitting. This advantage becomes more pronounced in complex industrial models that feature interacting concurrent components, urgent or broadcast synchronization channels, observer templates, and array-based constructs. In these settings, localized repairs by the baseline frequently fail subsequent global validation checks. Our empirical analysis shows that the baseline lacks explicit optimization to minimize edit magnitude, often generating extreme candidate bounds that violate admissibility. Additionally, the baseline struggles with regression verification over three to nine coupled properties because it does not analyze their corresponding counterexamples jointly.

MPTA-Repair achieves higher success rates due to two primary design advantages. First, multi-counterexample accumulation reduces overfitting to a single timed diagnostic trace, which typically exposes only one timing realization of an underlying fault. By accumulating and analyzing diagnostic traces from multiple operational modes, the framework synthesizes minimal joint edit sets that address the timing fault comprehensively. Second, relation-guided optimization complements precise MaxSMT objectives to improve candidate quality. By identifying diagnostic traces that share path fragments or target repair variables, the solver focuses on modifying numerical clock bounds close to the observed violations while penalizing large edit magnitudes. Successful repairs therefore typically require modifying only one or two critical clock bounds, which facilitates subsequent manual inspection and verification.

D. Discussion and Practical Lessons

The evaluation highlights several insights for applying automated repair to industrial timed automata. First, full-property regression testing is necessary. Because industrial system properties are interdependent, a localized fix designed solely to satisfy a liveness-like response monitor can inadvertently violate a separate safety monitor that constrains maximum waiting times. MPTA-Repair avoids these behavioral regressions by treating overall property restoration as a global objective. Second, admissibility is critical to measuring repair success. Without ensuring structural and functional equivalence, a solver might satisfy a specific timing property by disabling the intended functional behavior, such as preventing a critical

system action entirely. The strict admissibility check ensures that property satisfaction is not achieved at the expense of untimed functional behavior.

Our failure analysis reveals several limitations of the current repair space. Computational timeouts and unsupported model syntax account for only a portion of the unsuccessful repair attempts. In tightly coupled domains such as cardiac pacing, certain complex errors require simultaneous, consistent adjustments across both guards and invariants, which significantly expands the search space. Additionally, incomplete diagnostic trace coverage from the verification engine can occasionally lead to overfitting. Certain complex cases lacked any feasible candidate within the restricted clock-bound parameter space. This limitation suggests that repairing these models requires broader structural modifications, such as altering synchronization labels, adjusting clock resets, or modifying discrete data updates.

V. Conclusion

This paper presented **MPTA-Repair**, an automated workflow for multi-property and multi-counterexample clock-bound repair of industrial timed automata. To overcome the limitations of isolated single-trace fixes, the method accumulates diagnostic traces and exploits explicit relations among properties, counterexamples, and repairable bounds to guide a MaxSMT-based search. Each synthesized candidate is validated through full-property regression and an admissibility check based on untimed functional equivalence [9]. This design ensures that accepted repairs are system-level timing corrections that preserve the intended operational behavior.

Evaluations across benchmark and industrial datasets show that MPTA-Repair outperforms the iterative baseline, particularly in industrial systems with dense property interactions. The failure analysis further indicates that unsuccessful repairs often stem from the intrinsic limits of a strictly parametric clock-bound repair space.

Future work will primarily focus on extending the automated repair space beyond numerical clock bounds to include broader structural modifications, such as altering synchronization labels and redesigning discrete control logic. We also plan to investigate the theoretical and algorithmic relationship between candidate repair generation and admissibility verification. By formally integrating behavioral admissibility constraints into the early stages of search-space formulation, we aim to proactively prune functionally invalid candidates and thereby improve computational efficiency for large-scale industrial applications.

References

[1] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[2] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[3] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer," in *Proceedings of Formal Methods for Industrial Critical Systems*, 2021, pp. 176–193.

[4] Z. Jiang, M. Pajic, R. Alur, and R. Mangharam, "Modeling and verification of a dual chamber implantable pacemaker," in *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2012, pp. 188–203.

[5] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "Herschel-planck software schedulability analysis using UPPAAL," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, 2010, pp. 282–296.

[6] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[7] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[8] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[9] M. Koelbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," 2019, manuscript.

[10] ——, "TarTar: A timed automata repair tool," 2020, arXiv:2002.02760.

[11] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the ACM Symposium on Theory of Computing*, 1993, pp. 592–601.

[12] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, 2019.

[13] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," 2021, arXiv:2108.08018.

[14] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, 2011, pp. 504–509.

[15] R. Sebastiani and S. Tomasi, "Optimization modulo theories with linear rational costs," *ACM Transactions on Computational Logic*, vol. 16, no. 2, 2015.

[16] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proceedings of Computer Aided Verification*, 2000, pp. 154–169.

[17] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[18] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.

[19] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.

[20] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.